

# NumPy Practice Questions

**Q1.** Create one array covering 1 to 20 with a fixed step of 2, and another array covering the same range but with exactly 10 evenly spaced values. Without printing both fully, write a single line of code that tells you whether they have the same number of elements.

**Q2.** Create a float array and convert it to a 32-bit integer type. Predict what happens to values like 3.9 and 3.1 — do they round or truncate? Verify with code.

**Q3.** Create a 2D array of shape (3,4) filled entirely with zeros, then change only its data type to integer after creation — without recreating the array from scratch.

**Q4.** Create a 3D array of shape (2,3,4) filled with sequential numbers. Extract only the last row of every 2D block using slicing in a single line.

**Q5.** Given a 1D array of 20 elements, use negative indexing to extract every alternate element starting from the second-last element going backwards.

**Q6.** Create a 2D array (5x5). Using a condition-based selection, replace all values divisible by 3 with -1 — but only in the upper triangle of the matrix (don't touch the lower triangle or diagonal).

**Q7.** Select elements at positions (0,1), (2,3), (4,0) from a 5x5 array in a single line — without using a loop.

**Q8.** Create two arrays of different shapes — (3,1) and (1,4) — and add them together. Predict the resulting shape before running the code, and explain what's happening under the hood (this connects to a later module — figure out which).

**Q9.** Given an array of exam scores, use a comparison combined with a conditional array function to label scores as 'Pass' or 'Fail' (cutoff 40) — without using an if-else loop.

**Q10.** Create two arrays and demonstrate a case where a NumPy arithmetic function gives a different result than the standard + operator (hint: think about dtype casting or output storage).

**Q11.** Create a 1D array of 24 elements. Reshape it into (2,3,4), then collapse it back to 1D using two different approaches that behave differently with memory. Modify an element in the second version and check if the original array changes — explain why or why not.

**Q12.** Take a (4,3) array and attempt to reshape it into (3,3). Catch the resulting error using try-except and print a message explaining the exact mismatch in element count.

**Q13.** Create a 1D array of shape (5,) and convert it into shape (1,5) and separately into (5,1) by adding a new dimension — explain the difference in one line of comment.

**Q14.** Create a 2D array (4x3). Compute the average along each direction (rows vs columns) separately — without checking documentation, predict which one gives you 4 values and which gives 3, then verify.

**Q15.** Given an array of angles in degrees, convert them to radians manually (without using a built-in conversion function), then compute their sine values.

**Q16.** Create an array of floating-point numbers with both positive and negative decimals. Round them down and round them up separately, and explain (via printed output) why the results aren't simply 'rounding down' and 'rounding up' the way you might expect for negative numbers.

**Q17.** Create three 1D arrays of different lengths. Try stacking them vertically directly — catch the error, then pad the shorter arrays with zeros to make it work.

**Q18.** Given a 2D array (6x4), split it into 3 equal parts along rows, then try splitting it into 4 parts and explain what error occurs and why.

**Q19.** Insert a new row in the middle of a 2D array (not at the start or end), then remove the second column from the result — do both in a way that doesn't modify the original array.

**Q20.** Take a (3,4) array and a (4,) array. Add them and explain in a comment exactly how the smaller array gets stretched to match — no loops allowed.

**Q21.** Take a (3,1) array and a (1,3) array. Multiply them and predict the resulting shape before running — this one's a trick, the output isn't what most beginners expect.

**Q22.** Create two arrays of shapes (2,3) and (3,2). Try adding them directly, catch the resulting error, then fix it using a shape-changing operation so they can be added meaningfully.

**Q23.** Iterate through a 3D array using a single for loop (not nested), printing each element along with its full index position — combine both tasks in one loop, not two separate ones.

**Q24.** Iterate through an array and modify each element in-place (multiply by 2) using an approach other than a plain for loop — explain why a plain for loop won't let you do this correctly.

**Q25.** Create an array, take a slice of it (which shares memory with the original), modify the slice, and prove the original array changed. Then repeat using an approach that does NOT share memory, and prove the original stays unchanged — show both outcomes with code, not just explanation.

**Q26.** Reshape an array (this can either share memory or create an independent copy depending on layout) and use an appropriate attribute to determine which one occurred in your specific case.

**Q27.** Create two matrices A (3x2) and B (2x3) using sequential numbers. Compute A·B and B·A separately, and without printing shapes, predict which one will fail if you accidentally reverse the multiplication order — then fix it.

**Q28.** Generate a 3x3 matrix of random integers between 1 and 10 (seed = 42). Compute its determinant. Now multiply the entire matrix by 2 and compute the determinant again — before running the code, guess whether the new determinant is exactly double, and verify.

**Q29.** Create a 4x4 identity matrix and modify only its anti-diagonal (top-right to bottom-left) to contain 5s instead of 1s or 0s. Find its determinant and inverse — if the inverse throws an error, explain why using code (don't just state it).

**Q30.** Take a 3x3 matrix where one row is a linear combination of the other two (you decide the values). Try computing its inverse. Catch the resulting error using try-except and print a custom message identifying why it happened, using the determinant as proof.

**Q31.** Create two square matrices A and B of size 3x3. Compute their matrix product both ways ( $A \times B$  and  $B \times A$ ). Without using an array-equality helper function, write a one-line boolean check to verify whether matrix multiplication is commutative for these two matrices.

**Q32.** Find the eigenvalues and eigenvectors of a 3x3 matrix of your choice (non-symmetric). Reconstruct the original matrix back using only the eigenvalues and eigenvectors, and verify it matches the original using an approximate-equality check.

**Q33.** Solve the system of equations:  $2x + 3y - z = 5$ ,  $4x + y + 2z = 6$ ,  $-2x + 5y + 3z = 4$ . Then verify your solution manually by substituting the values back using matrix multiplication instead of plain arithmetic.

**Q34.** Create a 3x3 matrix with a determinant of exactly 0 (you must construct it, not guess). Try solving  $Ax = b$  for any b vector and explain (via code, catching the exception) why NumPy refuses to solve it.

**Q35.** Given a 2x2 matrix, compute its inverse manually using the determinant and adjoint formula (without a built-in inverse function), then verify your manual result against the built-in result using an approximate-equality check.

**Q36.** Create a 3x3 matrix A. Compute A raised to the power of 3 in two different ways — once using a direct power function and once using repeated matrix multiplication in a loop. Confirm both results match exactly.

**Q37.** Set a seed, generate 5 random integers between 1-100, then generate 5 more WITHOUT resetting the seed. Reset the seed and generate 5 again — prove which set matches which using code, not just visually comparing outputs.

**Q38.** Generate an array of 1000 values from a standard normal distribution. Without plotting, use the mean and standard deviation to verify it's approximately following a standard normal distribution ( $\text{mean} \approx 0$ ,  $\text{std} \approx 1$ ) — what tolerance would you consider 'approximately'?

**Q39.** Create an unsorted 2D array (4x4). Sort it along each of the two directions separately — predict which one sorts 'columns' and which sorts 'rows' before running (this trips people up).

**Q40.** Get the index order that would sort a 1D array, then use those same indices to sort a DIFFERENT array of the same length (like sorting names based on corresponding scores) — without directly sorting the second array.

**Q41.** Given a sorted array, find where a new value should be inserted to keep it sorted — test it with a value that already exists in the array and explain which position it returns (left or right) by default.

**Q42.** Given a dataset (array) of 50 random values, compute the 25th, 50th, and 75th percentiles. Without using a dedicated median function, verify that the 50th percentile equals the median.

**Q43.** Create two arrays of the same length representing two variables. Compute their correlation and explain (in a comment) what the diagonal values of the resulting matrix represent.

**Q44.** Generate a random dataset and bucket it into 5 equal-width bins with their frequencies. Without plotting anything, print out which bin has the highest frequency and its range.

**Q45.** Save a 2D array to disk, then load it back and modify the loaded version — prove the original saved file on disk is untouched by re-loading it again.

**Q46.** Save two different arrays into a single compressed file with custom names, then load the file and access each array individually using those names — without loading it in list/array mode.

**Q47.** Save an array with mixed formatting requirements (some columns as integers, some as floats) into a text file with a custom format specifier, then load it back and verify the dtypes match what you intended.